

< The (Linear) Kalman Filter 7/13 >

Kalman filter와 recursive LS의 차이는 Kalman filter는 static state가 아니라 움직이는 state vector에 대해 적용이 가능하다는 점이다.

Kalman 필터는 3가지 단계로 이루어진다.

① Prediction. (motion model)

: 동역학 모델을 바탕으로 다음 step k 의 state vector를 센서 없이 예측함.

$$N(\hat{x}_k, \hat{P}_k)$$

② Measurement (observation model)

: 센서를 사용해 step k 의 state를 측정함 (센서의 오차 발생).

$$N(y_k, R_k)$$

③ Fusion

: ①과 ②의 정보를 종합해서 k step의 가장 정확한 state vector를 확정짓음

$$N(\hat{x}_k, \hat{P}_k)$$

* motion model

$$x_k = F_{k-1} x_{k-1} + G_{k-1} u_{k-1} + W_{k-1}$$

noise

동역학 수식을 행렬 형태로
집어넣어

외부의 명령(u_{k-1})은 G_{k-1} 행렬로
state의 변화로 반영시킴.

$W_k \sim N(0, Q_k)$: 이상적인 물리법칙에 반해 현실에서 발생하는 외란에 의한
불확실성 2차원 반영.

* Measurement model

$$y_k = H_k x_k + V_k$$

Jacobian

noise

$$V_k \sim N(0, R_k).$$

: 센서의 측정 2차원 반영

센서 측정 값

$$y_k = 17 - x_1 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

1) Prediction

$$\check{x}_k = F_{k-1} \check{x}_{k-1} + G_{k-1} u_{k-1}$$

$$\check{P}_k = F_{k-1} \check{P}_{k-1} F_{k-1}^T + Q_{k-1}$$

2) Optimal Gain

$$K_k = \check{P}_k H_k^T (H_k \check{P}_k H_k^T + R_k)^{-1}$$

3) Correction

$$\hat{x}_k = \check{x}_k + K_k (y_k - H_k \check{x}_k)$$

→ Prediction과 measurement의 차
(innovation, 잔차)

$$\hat{P}_k = (I - K_k H_k) \check{P}_k$$

< Prediction, measurement, correction 의 수식 유도 7/13 >

1) Prediction

실제 움직임 : $x_k = Fx_{k-1} + Gu_{k-1} + w_{k-1}$

알려진 예측을 위해 식 전체에 기대값을 씌우면

$$\check{x}_k = F\check{x}_{k-1} + Gu_{k-1} + E[w_{k-1}]$$

▷ Gaussian distribution

$$E[w_{k-1}] = 0$$

$$= F\check{x}_{k-1} + Gu_{k-1}$$

▷ 직접 입력한 값이므로 자체가 항값

Prediction 오차: $\check{e}_k = \check{x}_k - x_k$

$$= (F\check{x}_{k-1} + Gu_{k-1}) - (F\check{x}_{k-1} + Gu_{k-1} + w_{k-1})$$

$$= F\check{x}_{k-1} - \check{x}_{k-1} - w_{k-1}$$

$$= F\check{e}_{k-1} - w_{k-1}$$

$$\check{P}_k = E[\check{e}_k \check{e}_k^T] = F\check{P}_{k-1}F^T + Q_{k-1}$$

2) Measurement

측정 모델: $y_k = Hx_k + v_k$

Innovation ($e_k = v_k$) = $y_k - H\check{x}_k$: 예측과 현실의 오차

$$\hat{x}_k = \check{x}_k + K_k(y_k - H\check{x}_k)$$

최종 오차 (항값 차와 각변편차는 아닌 $\hat{x}_k$) : $\hat{e}_k = \hat{x}_k - x_k$

$$\hat{e}_k = -\tilde{x}_k + (\tilde{x}_k + K_k(y_k - H_k \tilde{x}_k))$$

$$= -\tilde{x}_k + (\tilde{x}_k + K_k(H_k \tilde{x}_k + v_k - H_k \tilde{x}_k))$$

$$= (\tilde{x}_k - \tilde{x}_k) + K_k(H_k(\tilde{x}_k - \tilde{x}_k) + v_k)$$

$$= \check{e}_k - K_k(H_k \check{e}_k - v_k)$$

$$= (I - K_k H_k) \check{e}_k + K_k v_k$$

단일 데이터가 아니라 여러번 (1000번 정도) 무한번 시도 했을 때의 상황에 대응해서 해석하겠다는 의미로 기댓값을 계산.

$$\hat{P}_k = E[\hat{e}_k \hat{e}_k^T]$$

$$= (I - K_k H_k) \check{P}_k (I - K_k H_k)^T + K_k R_k K_k^T$$

$$\mathcal{L}(x) = \text{Trace}(\hat{P}_k)$$

$\hat{P}_k$ 의 diagonal 성분은 분산의 위치하게 된다. 이때 분산은 예측과 센서 분포의 모든 반영된 σ_m^2

$$\frac{\partial (\text{Trace}(\hat{P}_k))}{\partial K_k} = 0 \text{ 이 되도록 하는 } K_k = \check{P}_k H_k^T (H_k \check{P}_k H_k^T + R_k)^{-1}$$

$$\rightarrow \hat{P}_k = (I - K_k H_k) \check{P}_k$$

< Kalman Filter and Bias BLUES 7/14 >

Kalman Filter에서 k step에 대해 무한번 시뮬레이션을 돌린다면 그 결과는 다음과 같다.

$$E[\hat{e}_k] = 0, E[V] = 0, E[W] = 0 \oplus \text{white, uncorrelated noise를 전제로}$$

$$\rightarrow E[\check{e}_k] = 0, E[\hat{e}_k] = 0$$

* consistency

① unbiased : $E[\hat{e}_k] = 0$

② 공분산 일치 : $E[\hat{e}_k \hat{e}_k^T] = \hat{P}_k$

$\hat{e}_k = \hat{z}_k - z_k$ 를 형식에서 발생하는 실제 오차를 의미. $\hat{P}_k$ 는 v_k, w_k 를 고려한 값으로 정해 유요한 양과 음 내부 불확실성. 두 계산이 일치하면 Consistence, 가장 이상적인 상황

1) Overconfident

실제 오차는 매우 큰데 양과 음 내부의 P_k 가 작으면, 센서로부터 새로운 데이터를 받아들이지 않아 (불확실성 P_k 가 작으므로) 참값과 추정값이 영원히 일치함.

2) underconfident

실제 오차는 매우 작는데 P_k 가 상대적으로 큰 경우. 안전한 하지만 센서의 노이즈에 크게 휘둘려서 최적의 estimator 성능을 기대하기는 어렵다

3) consistent

: 실제 오차를 P_k 가 완벽하게 예측해냄. 자신의 불확실성을 현실하게 알고 있으므로 Prediction과 measurement의 가중치를 이상적으로 설정 가능